

COMP0472 - Sistemas Operacionais - 2026.2

RAG sobre PDF com Modelos de Linguagem Locais

Trilha C - AV1

Alex Felipe Santos Melo - 202200025395

Prof. Glauco Estácio Gonçalves - UFS

Contexto e Objetivo

Pergunta norteadora

Como a camada de aplicacao, o modelo, a quantidade de parametros, a quantizacao e a configuracao de execucao afetam processos, threads, uso de CPU, memoria, armazenamento e responsividade de um sistema local de IA generativa?

O que foi feito

1. Instalar e executar um sistema RAG local com Ollama + DeepSeek-R1
2. Medir TTFT, tempo total de resposta e uso de RAM em 6 configuracoes
3. Analisar chamadas de sistema com strace durante a inferencia em CPU

Ambiente e Arquitetura RAG

Ambiente de execucao

WSL2 · Ubuntu 22.04

Windows 11

AMD Ryzen 7 7735HS

7.8 GB RAM (limite WSL2)

Ollama 0.6.2

inferencia em CPU · sem GPU

DeepSeek-R1:7b

4.7 GB · modelo principal

DeepSeek-R1:1.5b

1.1 GB · experimento C3-A

nomic-embed-text

embeddings RAG

Pipeline RAG

PDF (upload)

Chunks (9 fragmentos)

nomic-embed-text

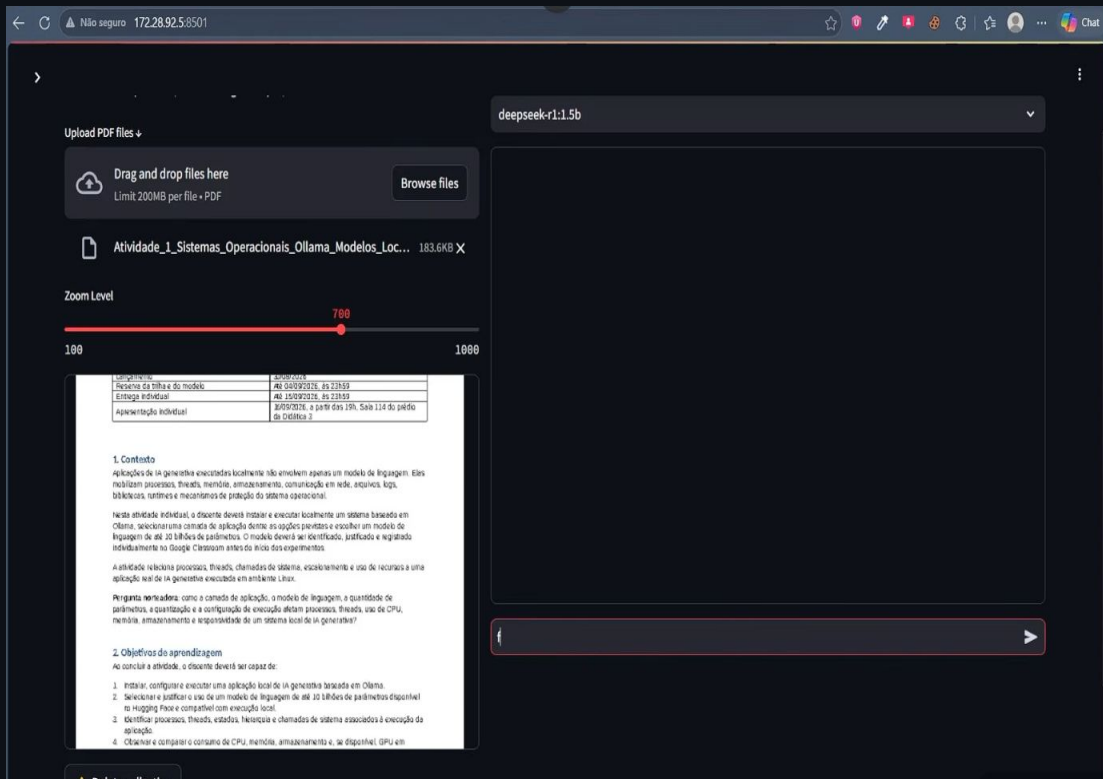
ChromaDB

Query + Contexto

DeepSeek-R1:7b

Resposta

Sistema RAG em Funcionamento



183 KB

PDF indexado

9

chunks extraídos

0

APIs externas

Inferencia 100% local

Metodologia de Medicao

medir.sh

```
# API REST do Ollama
curl http://localhost:11434/api/generate

# Captura TTFT e tempo total
.prompt_eval_duration, .total_duration

# RAM do processo
ps -o rss= -p $(pgrep ollama)
```

Protocolo experimental

4 rodadas por configuracao
Media das 2 ultimas (tail 2) por consistencia
Resultados salvos em CSV por rodada

Metricas coletadas

TTFT

Time to First Token

segundos

Total

Resposta completa

milissegundos

RAM

Memoria do processo

MB via ps

Configuracoes dos Experimentos

C1-A

Baseline - query curta

Referencia: prompt curto, contexto padrao de 8192 tokens

C1-B

Baseline - query longa

Prompt extenso, gera resposta muito maior pelo modelo

C2-A

Requisicoes concorrentes

2 chamadas simultaneas a API do Ollama

C2-B

Pipeline RAG ativo

ChromaDB + recuperacao de contexto ligado

C3-A

Modelo menor (1.5b)

deepseek-r1:1.5b (1.1 GB) substitui o modelo 7b

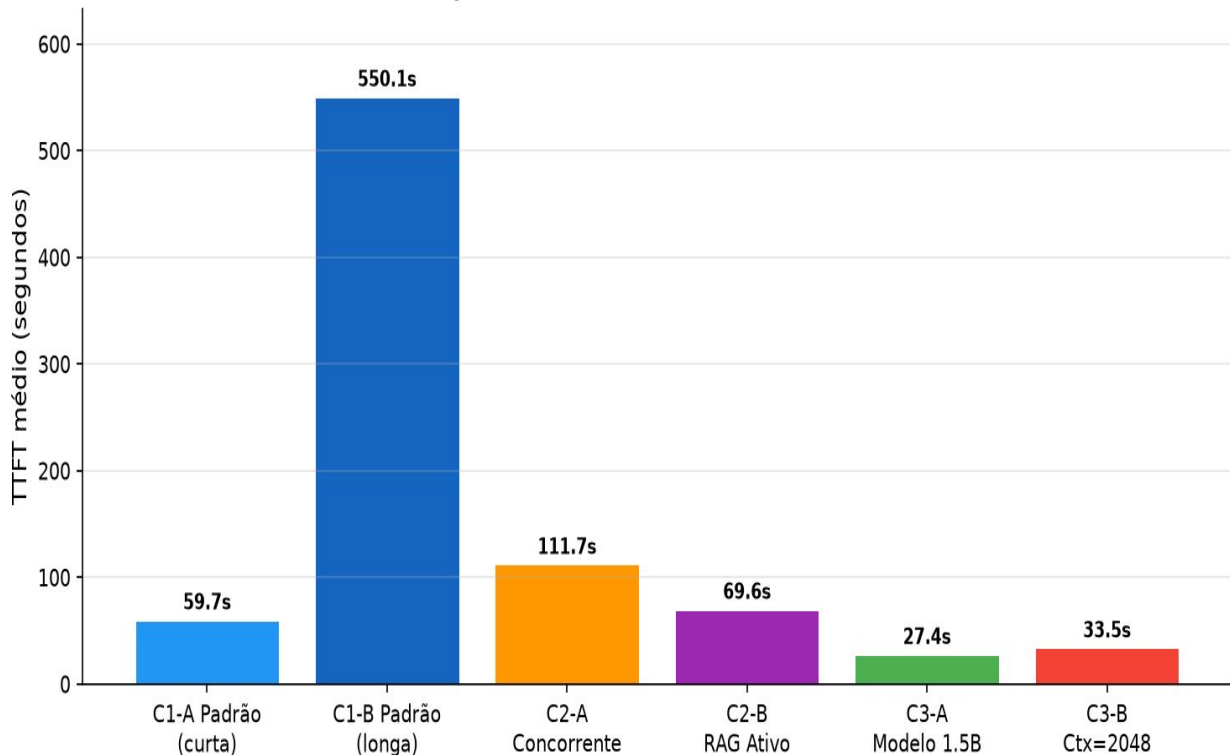
C3-B

Contexto restrito

num_ctx=2048 no lugar de 8192, janela de contexto reduzida

Resultados: Latencia (TTFT)

Latência de inferência (TTFT) por configuração
DeepSeek-R1 em CPU - WSL2, 7.8 GB RAM



59.7s

Baseline C1-A

550s

C1-B query longa

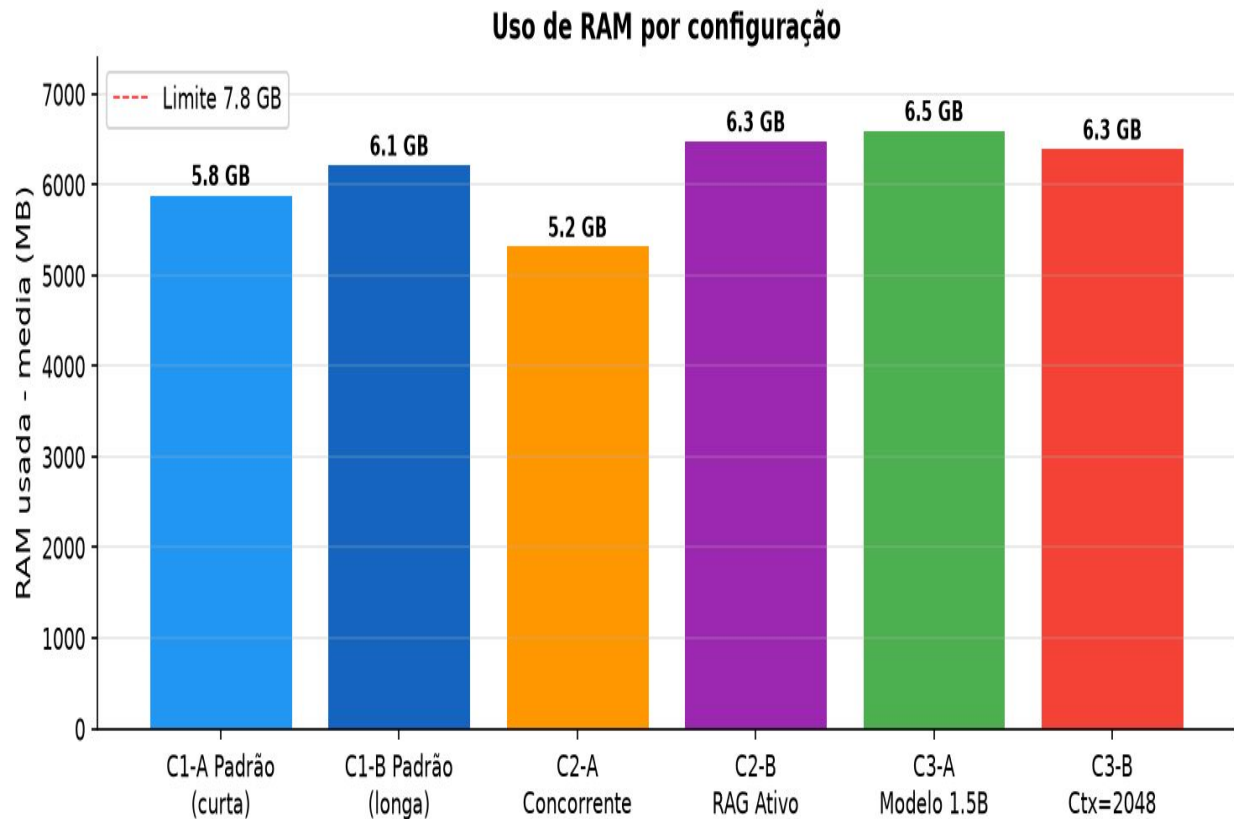
27.4s

C3-A modelo 1.5b

-54%

reducao de TTFT

Resultados: Uso de RAM



5.2 GB

minimo (C2-A)

6.5 GB

maximo (C3-A)

~1.3 GB

variacao total

7.8 GB

limite WSL2

Análise com strace

76%

das syscalls são futex

O que é futex?

Permite que threads gerenciem bloqueios e desbloqueios diretamente no espaço de usuário (userspace) na ausência de contenção, recorrendo ao kernel apenas quando há necessidade de suspender a execução.

Interpretação

O modelo emprega múltiplas threads para executar tarefas em paralelo de forma eficiente.

Sincronização em CPU: Em ambientes orientados puramente à CPU (CPU-only), as threads realizam sincronizações frequentes utilizando o futex para garantir a consistência das operações.

Gargalo em CPU-only: sincronização entre threads do modelo (futex)

Não é I/O de disco ou rede: é coordenação computacional entre threads

Conclusão

- 01** Modelo menor (1.5b) reduziu o TTFT em 54%: maior impacto positivo observado
- 02** Contexto gerado é o principal fator de latência: C1-B chegou a 550 segundos
- 03** Uso de RAM estável em todas as configurações: modelo carregado domina a memória
- 04** strace confirma: gargalo em futex (threads), não em I/O de disco ou rede